

API (INTERFACE DE PROGRAMAÇÃO DE APLICATIVOS): CONCEITOS, ARQUITETURAS E APLICAÇÕES

1 INTRODUÇÃO

Uma API (Interface de Programação de Aplicativos) é um conjunto de regras e protocolos que permite a comunicação entre diferentes aplicativos de software. Em vez de os desenvolvedores programarem funcionalidades do zero, as APIs permitem reutilizar serviços e dados de outras aplicações. O conceito fundamental de uma API é permitir a integração entre sistemas, onde um fornece informações e serviços sem que o sistema consumidor precise conhecer os detalhes internos de implementação.

De forma prática, uma API funciona como o garçom de um restaurante: o cliente (aplicação) escolhe opções em um menu, o garçom (API) leva o pedido até a cozinha (servidor) e, ao final, o garçom entrega o prato (resultado) de volta ao cliente.

O uso de APIs permite o desacoplamento de um sistema, viabilizando que um único back-end se comunique com múltiplos front-ends, o que reduz custos e o tempo de desenvolvimento.

2 CONCEITOS E FUNDAMENTOS

2.1 Comunicação Cliente-Servidor

A comunicação ocorre através de um fluxo de Solicitação e Resposta (Request-Response). O cliente envia uma requisição com um identificador (URI), e a API atua como uma ponte invisível até o servidor, que processa e devolve a informação.

2.2 Web Services

Quando as APIs utilizam o protocolo HTTP para comunicação na internet, elas são chamadas de Web Services. Esses serviços trocam dados, na maioria das vezes, utilizando os formatos XML ou JSON, tornando a comunicação independente da linguagem de programação do cliente ou do servidor.

2.3 Front-end vs Back-end

As APIs operam majoritariamente no back-end (a lógica e o processamento da aplicação), servindo como ponto de acesso padronizado para o front-end (a interface visual e interativa do usuário).

3 TIPOS DE APIS NA WEB

3.1 APIs Abertas (Públicas)

Estão disponíveis para desenvolvedores externos acessarem funcionalidades livremente pela internet, definindo seus próprios endpoints.

3.2 APIs de Parceiros

São restritas a parceiros comerciais estratégicos da empresa, exigindo processos de integração e credenciais de login para acesso.

3.3 APIs Internas (Privadas)

São mantidas ocultas do público e usadas exclusivamente dentro da própria organização para integrar sistemas internos e aumentar a produtividade.

3.4 APIs Compostas

Permitem acessar e combinar dados de múltiplos serviços ou endpoints em uma única chamada, sendo muito úteis em arquiteturas de microserviços.

4 ARQUITETURAS E PROTOCOLOS

4.1 REST (Representational State Transfer)

É um estilo arquitetural criado por Roy Thomas Fielding, que se tornou o padrão mais popular por ser leve e flexível. A arquitetura REST possui restrições como:

- **Client-Server:** separação entre quem solicita e quem provê o serviço.
- **Stateless:** o servidor não guarda informações (estado) do cliente entre as requisições.
- **Cache:** respostas podem ser marcadas como armazenáveis para otimizar o tráfego de rede.
- **Uniform Interface:** a comunicação com os endpoints ocorre de forma padronizada para qualquer dispositivo.

- **Layered System:** o sistema atua em camadas, e um componente só enxerga a camada imediatamente conectada a ele.

4.2 SOAP (Simple Object Access Protocol)

É um protocolo baseado em XML. Embora mais rígido e complexo de implementar do que o REST, possui forte tipagem e recursos integrados de alta segurança e confiabilidade, sendo preferido em sistemas governamentais e financeiros.

4.3 GraphQL

Uma linguagem de consulta criada pelo Facebook que oferece alta flexibilidade. Diferente do REST, que pode trazer dados excessivos, o GraphQL permite que o cliente requisite e receba exatamente as informações específicas que precisa em uma única chamada.

4.4 RPC (Remote Procedure Call)

Protocolos que permitem chamar rotinas em servidores como se fossem locais, englobando padrões como XML-RPC, JSON-RPC e gRPC.

5 MÉTODOS E CÓDIGOS HTTP

Na arquitetura REST, as requisições utilizam verbos do protocolo HTTP para realizar ações específicas e retornam códigos de status.

5.1 Verbos HTTP

- GET (solicita dados).
- POST (cria/submete dados).
- PUT (edita/substitui informações).
- DELETE (remove um recurso).

5.2 Status de Resposta

- 200 OK: sucesso genérico.
- 201 Created: sucesso após criar recurso via POST.
- 204 No Content.
- 400 Bad Request: sintaxe da requisição inválida.
- 404 Not Found: recurso não encontrado.
- 500 Internal Server Error: falha interna.

6 FERRAMENTAS DE DESENVOLVIMENTO E BANCO DE DADOS

6.1 Linguagens e Frameworks

O desenvolvimento pode ser feito em Python, usando frameworks leves voltados para alta performance como o Falcon, ou em Java, utilizando o ecossistema e módulos do Spring Boot.

6.2 Mapeamento Objeto-Relacional (ORM)

Ferramentas que facilitam a integração do código com bancos de dados relacionais (SQL), como o SQLAlchemy no Python e o Spring Data JPA no Java.

6.3 Testes e Versionamento

Softwares como o Postman são usados para testar manualmente os envios de requisições para a API. O código pode ser controlado e hospedado no GitHub, e plataformas como Docker criam containers isolados que garantem que a API funcione de forma consistente em qualquer sistema.

7 SEGURANÇA EM APIS

A proteção da API envolve estratégias de Autenticação (verificar quem é o usuário) e Autorização (o que ele tem permissão para acessar).

Para proteção de credenciais no banco de dados, utiliza-se a estratégia de Hash e Salt, que transforma a senha em um valor aleatório fixo, impedindo que seja armazenada em texto simples.

As APIs modernas utilizam Tokens (como o JWT) para validar sessões temporárias com segurança.

8 EXEMPLOS PRÁTICOS DE USO

- **WeatherAPI:** uma API externa muito utilizada para obter informações meteorológicas (temperatura, umidade, vento) em tempo real, fornecendo os dados em JSON.
- **Logins Universais:** recurso que permite a um usuário se cadastrar em novos sites utilizando suas contas do Google, Apple ou Facebook.

- **Comparações de Viagens:** sites que agregam e exibem simultaneamente valores de voos de dezenas de companhias aéreas.
- **Aplicativos de Navegação:** softwares que integram mapas e radares de trânsito em tempo real.
- **Internet das Coisas (IoT):** integração para automação e comandos remotos, como uma geladeira inteligente se comunicando com o celular.

9 CONCLUSÃO

As APIs consolidam-se como ferramentas indispensáveis para a modernização dos sistemas computacionais na economia digital. Elas funcionam como pontos centrais que promovem a escalabilidade, facilitam a automação de processos industriais ou pessoais e viabilizam a comunicação entre dispositivos de naturezas diferentes. Ao permitirem que componentes da infraestrutura tecnológica evoluam de forma independente, as APIs garantem flexibilidade e otimização para que diferentes plataformas e clientes operem sobre bases unificadas, seguras e eficientes.

REFERÊNCIAS

BONIN, Douglas Vinicius Caldas. **Implementação de uma API para visualização e interação com objetos de aprendizagem para ferramenta de autoria FARMA**. 2024. Trabalho de Conclusão de Curso (Tecnologia em Sistemas para Internet) - Universidade Tecnológica Federal do Paraná, Guarapuava, 2024. Disponível em: <https://repositoriocopia.utfpr.edu.br/jspui/handle/1/34010>. Acesso em: 15 ago. 2026.

BORGES, Hudson S. *et al.* **APIMiner 2.0: Recomendação de Exemplos de Uso de APIs usando Regras de Associação**. Departamento de Ciência da Computação, Universidade Federal de Minas Gerais (UFMG) e Universidade Federal de Lavras (UFLA), [s.d.]. Disponível em: https://scholar.google.com.br/scholar?hl=pt-BR&as_sdt=0%2C5&q=APIMiner+2.0%3A+Recomendac%C2%B8%CB%9C+ao+de+Exemplos+de+Uso+de+APIs+usando+Regras+de+Associac%C2%B8%CB%9C+ao&btnG=. Acesso em: 15 ago. 2026.

GALINDO JUNIOR, Edemilton Alcides; ROCHA, Romeu Dias; MACIEL, Ronierison de Souza. Desenvolvimento de API REST com Spring Boot. **Revista Científica do UniRios**, v. 2021.1, p. 499-525, 2021. Disponível em: <https://www.publicacoes.unirios.edu.br/index.php/revistarios/article/view/102>. Acesso em: 15 ago. 2026.

GONÇALVES, Vitor Tavares. **Introdução à criação de APIs: conceitos fundamentais e implementação em linguagem Python**. 2025. Monografia (Engenharia de Controle e Automação) - Escola de Minas, Universidade Federal de Ouro Preto, Ouro Preto, 2025. Disponível em: <https://monografias.ufop.br/handle/35400000/8436>. Acesso em: 15 ago. 2026.

GOODWIN, Michael. O que é uma API (interface de programação de aplicativos)?. **IBM Think**, [s.d.]. Disponível em: <https://www.ibm.com/br-pt/think/topics/api>. Acesso em: 15 ago. 2026.

JASSE, Ermínio Pita. **Application programming interface - API para integração de dados em agricultura de precisão**. 2017. 45 f. Dissertação (Mestrado em Tecnologias Computacionais para o Agronegócio) - Universidade Tecnológica Federal do Paraná, Medianeira, 2017.

OLIVEIRA, Paulo Henrique Cardoso de. **Desenvolvimento de um gerador de API REST seguindo os principais padrões da arquitetura**. 2014. 88 f. Trabalho de Curso (Bacharelado em Sistemas de Informação) - Centro Universitário Eurípides de Marília, Fundação de Ensino "Eurípides Soares da Rocha", Marília, 2014. Disponível em: <https://aberto.univem.edu.br/handle/11077/1008>. Acesso em: 15 ago. 2026.

WINCK, Arthur Pellenz. **Desenvolvimento de uma API para integração e manipulação de dados do portal Conecta: facilitação no acesso a dados governamentais**. 2025. Trabalho de Conclusão de Curso (Bacharelado em Ciências da Computação) - Universidade Federal de Santa Catarina, Florianópolis, 2025. Disponível em: <https://repositorio.ufsc.br/handle/123456789/266481>. Acesso em: 15 ago. 2026.